

CARLY

Analyse de ton fichier config/database.php

✓ Tes points forts

Simplicité : Le code est clair et facile à comprendre pour un débutant. Tu vas à l'essentiel pour établir la connexion.

Utilisation de PDO : C'est le bon choix technologique. PDO est robuste et permet de changer de type de base de données plus facilement si nécessaire.

Gestion d'exception : Tu as bien configuré ATTR_ERRMODE sur ERRMODE_EXCEPTION, ce qui te permettra de voir précisément ce qui ne va pas lors de tes requêtes SQL.

Tes points à améliorer

Encapsulation : Ta variable \$pdo est déclarée dans l'espace global. Cela peut devenir risqué si ton application grandit, car n'importe quel autre script pourrait écraser cette variable par erreur.

Sécurité des messages : L'affichage direct de \$e->getMessage() est pratique pour toi pendant que tu codes, mais c'est une faille de sécurité en production car cela peut révéler ton nom d'utilisateur ou ta structure de base de données à un pirate.

Charset manquant : Tu n'as pas précisé le jeu de caractères (UTF-8) dans ton DSN. Tu risques d'avoir des soucis d'affichage si tes contacts ont des noms avec des accents ou des caractères spéciaux.

Analyse de ton models/Contact.php

L'erreur de cohérence (Le point bloquant)

Dans ton `__construct()`, tu écris `$db = new Database();`. Or, dans le fichier `database.php`, il n'y a pas de classe `Database`, seulement une variable `$pdo`. Ton code va donc générer une erreur fatale (Fatal error: Class 'Database' not found).

✔ Tes points forts

Utilisation de l'objet : Tu as choisi de créer une classe avec des propriétés privées (`$conn`, `$table`). C'est une excellente habitude pour protéger tes données.

Requêtes préparées : Tu maîtrises bien l'utilisation des marqueurs nommés (`:name`, `:email`) dans tes méthodes `create` et `delete`. C'est parfait pour la sécurité contre les injections SQL.

Organisation : L'utilisation d'une propriété `$table` permet de rendre ton code plus flexible : si tu changes le nom de ta table en base de données, tu n'as qu'une seule ligne à modifier.

Tes points à améliorer

La faille de sécurité dans `getAll()` : Attention ! Ta méthode `getAll()` utilise `$this->conn->query()`. Si jamais tu décidais d'ajouter un filtre (un `WHERE`) en utilisant une variable dans cette requête sans passer par `prepare()`, tu créerais une faille de sécurité. Prends l'habitude de tout préparer.

Inclusion de fichier : Comme pour les autres, le `require_once` avec un chemin relatif peut poser problème selon l'endroit d'où ton script est lancé.

Analyse de ton ContactController.php

✓ Tes points forts

Structure Objet : Tu as bien instancié ton modèle dans le constructeur. C'est beaucoup plus évolutif.

Méthodes claires : Tes méthodes `list()`, `add()` et `delete()` sont courtes et faciles à lire. Elles se concentrent sur une seule tâche à la fois.

Gestion du POST : Tu vérifies bien la méthode de la requête (`$_SERVER['REQUEST_METHOD'] === 'POST'`) avant de traiter les données, ce qui évite des exécutions accidentelles.

Tes points à améliorer

Absence de validation d'email : Tu vérifies si le champ est vide (`!empty()`), mais tu ne vérifies pas si l'utilisateur a saisi un format d'email valide. Si j'écris "n'importe quoi" dans le champ email, ton code l'acceptera.

Sécurité de la suppression : Tu récupères `$_GET['id']` et tu l'envoies directement au modèle. Même si ton modèle utilise des requêtes préparées (ce qui est bien), il est toujours préférable de vérifier que l'ID est un nombre entier dans le contrôleur.

Expérience utilisateur (UX) : Tu rediriges systématiquement vers la liste, mais tu ne passes aucun message de confirmation (ex: "Contact ajouté avec succès"). L'utilisateur risque de se demander si son action a bien fonctionné.

Analyse de ton fichier `views/contacts.php`

✓ Tes points forts

Sécurité (Protection XSS) : Tu as parfaitement utilisé `htmlspecialchars()` pour l'affichage du nom et de l'email. C'est le point le plus important pour la sécurité d'une vue, et tu as réussi le test.

Accessibilité et Structure : Ton HTML est valide et bien structuré. L'utilisation d'un tableau (`<table>`) est tout à fait appropriée pour l'affichage de données de type répertoire.

Expérience Utilisateur (UX) : Tu as intégré la confirmation de suppression en JavaScript directement dans le lien. C'est une sécurité simple mais indispensable.

Utilisation du `foreach` : Ta boucle est propre et utilise la syntaxe alternative de PHP (`foreach : ... endforeach;`), ce qui est la norme recommandée dans les fichiers de vue pour une meilleure lisibilité.

Tes points à améliorer

Design (UI) : C'est le point où tu perds des points par rapport aux autres. Ton fichier ne contient aucun CSS. L'interface est très brute ("brutalist"), ce qui est fonctionnel mais manque d'attrait visuel pour un utilisateur final.

Gestion des messages : Il n'y a aucun affichage pour les messages d'erreur ou de succès. Si l'ajout échoue (par exemple à cause de l'erreur de connexion dans ton modèle), l'utilisateur ne saura pas pourquoi.

Cohérence des liens : Tes liens pointent vers `index.php?action=...`, ce qui confirme que tu as bien compris le rôle du routeur (le fichier `index.php` au sommet de ton projet).

Fiche d'Évaluation : Projet Répertoire MVC

Étudiant : Carly

Projet : Gestionnaire de Contacts (Structure Standard)

Note Globale : 14 / 20

Détail de l'Évaluation

Critère	Note	Observations
Architecture MVC	3,5 / 5	Le découpage est bon, mais attention à la cohérence : ton modèle appelle une classe Database qui n'existe pas dans ta config.
Sécurité & Rigueur	4,5 / 5	Très bien. Requêtes préparées en back-end et protection XSS en front-end. Il manque juste la validation du format d'email.
Qualité du Code (PHP)	4 / 5	Code très structuré et orienté objet. C'est une base solide pour des projets plus complexes.
Interface & UX (UI)	2 / 5	L'interface est purement fonctionnelle. Un minimum de CSS aurait permis de valoriser ton travail technique.